# How Sparse Are Useful Transformer Representations?

Random vs. Data-Driven Dimension Selection for Text Classification

Leo Cheng
leocheng@g.harvard.edu

Chufei Peng
chufeipeng@g.harvard.edu

Krystal Qiu
krystalqiu@g.harvard.edu

Jiangqi Wu
jiangqiwu@g.harvard.edu

November 4, 2026

## 1 Introduction

### 1.1 Background and Motivation

Pretrained representations such as BERT [1] support many language tasks, but an individual task may require only part of the information they encode. A representation's dimension is determined by the encoder architecture rather than the requirements of a particular prediction problem. This motivates asking whether a substantially smaller representation can preserve the performance of the full embedding on a specific downstream task.

The question has both practical and statistical relevance. Smaller representations can reduce downstream storage and classifier computation. More importantly for this project, comparing ways to reduce a representation helps characterize how predictive information is distributed: is sentiment recoverable from many arbitrary coordinate subsets, or does retaining it depend on identifying particular coordinates or directions? We investigate this distinction using frozen, 768-dimensional BERT embeddings, so that changes in classification performance can be studied without simultaneously updating the encoder.

### 1.2 Related Work

Neuron-level analyses provide evidence that the concentration of useful information depends on the property being predicted. Durrani et al. [2] study morphology, syntax, and semantics in pretrained language models and identify small neuron subsets that predict linguistic properties. They find that morphology is more localized than syntax and that information in BERT is more distributed than in XLNet. These findings motivate examining coordinate subsets while avoiding the assumption that every task admits an equally sparse representation.

Dalvi et al. [3] examine redundancy more directly, distinguishing general redundancy from task-specific redundancy in BERT and XLNet. Their layer- and neuron-level analyses show that selected subsets can retain much of downstream performance. This work motivates our focus on predictive sufficiency: a representation may encode many properties, while only some of them are needed for a given task. Our study considers the narrower setting of pooled review embeddings and compares coordinate selection with projection at matched output dimensions.

Classical statistical methods provide complementary approaches to this comparison. The lasso introduced by Tibshirani [4] motivates using $\ell_1$-regularized logistic regression for supervised coordinate selection. PCA provides

an unsupervised projection based on explained variance [5]. Their different objectives motivate asking whether label-informed selection preserves sentiment information more effectively than variance-based projection. Random coordinate subsets provide a reference for reduction without informed selection. Together, these approaches connect the representation studies above to feature selection and dimension reduction; their detailed implementations are given in the methodology section.

### 1.3 Research Questions and Scope

We ask how closely smaller representations approach the full-dimensional baseline, whether supervised coordinate selection outperforms random selection at the same size, and how both compare with PCA. Repeating random selection also allows us to assess sensitivity to the particular coordinates retained.

We evaluate these questions on Amazon Review Polarity [6], using balanced samples of 20,000 training and 10,000 test reviews. Its positive and negative sentiment labels and predefined splits provide a common setting for evaluating predictive loss on held-out data. All methods use the same frozen BERT embeddings and the same $\ell_2$-regularized logistic regression classifier family. Our contribution is an empirical comparison of established methods in this setting. Any evidence of sparse predictive sufficiency concerns this task and embedding basis; it does not establish universal sparsity or reduce the cost of computing the original BERT embeddings.

## 2 Data and Methodology

### 2.1 Dataset and preprocessing

We use Amazon Polarity, a binary sentiment dataset of product reviews, accessed through `fancyzhx/amazon_polarity` on Hugging Face.[1] One- and two-star reviews are labeled negative ($y = 0$), while four- and five-star reviews are labeled positive ($y = 1$). Three-star reviews are excluded.

We preserve the original training and test splits. To construct smaller samples, we first draw candidate pools of 60,000 training reviews and 30,000 test reviews without replacement using NumPy seed 0. We combine each review's title and content, adding a period after the title when needed, normalize whitespace, and remove empty texts and exact duplicates. We also remove test-pool texts that appear in the cleaned training pool. This removes exact text overlap but does not identify near-duplicates.

Using sampling seed 0, we draw balanced samples of 20,000 training reviews and 10,000 test reviews from the cleaned pools. Table 1 summarizes the samples. Word counts are based on whitespace-separated words.

**Table 1:** Summary of the preprocessed reviews.

| Split | Reviews | Negative | Positive | Median words | 95th pct. words |
|---|---|---|---|---|---|
| Train | 20,000 | 10,000 | 10,000 | 70 | 161 |
| Test | 10,000 | 5,000 | 5,000 | 69 | 160 |

### 2.2 Frozen BERT representation

We extract embeddings using `bert-base-uncased` [1], with all model parameters frozen. The model runs in evaluation mode with gradient computation disabled. Reviews are tokenized in batches of 64, padded within each batch, and truncated to a maximum of 256 tokens, including special tokens. Before truncation, median token counts are 91 for training reviews and 90 for test reviews. Only 0.17% of training reviews and 0.18% of test reviews exceed the limit.

For review $i$, we average the final-layer token

[1] `https://huggingface.co/datasets/fancyzhx/amazon_polarity`

vectors using the attention mask:

$$x_i = \frac{\sum_{t=1}^{T_i} a_{it} h_{it}}{\sum_{t=1}^{T_i} a_{it}} \in \mathbb{R}^{768},$$

where $h_{it}$ is the token representation and $a_{it}$ is its attention-mask value. Padding tokens are excluded from the average, while special tokens are included. The training and test embedding matrices have sizes 20,000×768 and 10,000×768, respectively. Extraction uses float16 autocasting on the GPU, and embeddings are stored as float32 arrays and reused across experiments.

## 2.3 Exploratory analysis

We examine training-review token lengths overall and by label. For the training embeddings, we summarize coordinate standard deviations, pairwise Pearson correlations, and correlations between individual coordinates and the sentiment label. We also examine cumulative explained variance from PCA on standardized training embeddings and label correlations of the first 50 component scores. These summaries describe how variation and marginal sentiment association are distributed across the representation. The EDA result will be presented in the result section.

## 2.4 Representation reduction

Before selection or projection, each embedding coordinate is standardized using its training-set mean and standard deviation. The same transformation is applied to the test embeddings. We compare reduced sizes $k \in \{25, 50, 100, 200, 400\}$ with the full 768-dimensional baseline.

**Full representation.** The baseline retains all 768 coordinates and provides the reference for measuring performance losses.

**Random coordinate selection.** For each $k$, we select $k$ coordinates uniformly without replacement and repeat this procedure 20 times. One NumPy random generator initialized with seed 42 produces the draws sequentially. Subsets are not constrained to be nested across sizes. A separate classifier is fitted for each subset.

**Supervised L1 selection.** We fit L1-regularized logistic regression to the standardized training embeddings using the `liblinear` solver and a maximum of 3,000 iterations. The regularization path contains 17 logarithmically spaced values of the inverse regularization parameter $C$ from $10^{-4}$ to 1.

For each $k$, we choose the smallest $C$, corresponding to the strongest penalty on the path, that yields at least $k$ nonzero coefficients. We retain the $k$ coordinates with the largest absolute coefficients. Every target size is reached in the full run; Table 2 gives the selector settings. The L1 model selects coordinates, and a separate L2 classifier produces the final predictions.

**Table 2:** L1 selector settings. Nonzero counts are measured before retaining the top $k$ coordinates. Values of $C$ are rounded.

| $k$ | Selector $C$ | Nonzero coefficients |
|---|---|---|
| 25 | 0.00100 | 36 |
| 50 | 0.00178 | 57 |
| 100 | 0.00562 | 128 |
| 200 | 0.0178 | 250 |
| 400 | 0.0562 | 408 |

**Principal component analysis.** We fit PCA to the standardized training embeddings, retaining 400 components, and use the first $k$ component scores for each reduced classifier. Training-derived centering and loadings are used to transform the test embeddings. We also record cumulative explained variance at each size.

PCA components combine information from the original coordinates. Equal $k$ therefore means equal downstream input dimension, but does not imply equal dependence on the original coordinates or equal transformation cost.

## 2.5 Classifier and tuning

All representations use the same classifier pipeline: `StandardScaler` followed by L2-regularized logistic regression, with a maximum of 3,000 iterations. We tune the inverse regularization parameter $C$ over 11 logarithmically spaced values from $10^{-4}$ to 10, using five-fold stratified cross-validation with shuffled folds and seed 42.

The value with the highest mean validation accuracy is selected, and the pipeline is refitted on the complete training set. Each configuration is evaluated on the same held-out test set, which is excluded from fitting and tuning. Representation selection and PCA are performed before classifier cross-validation, so the coordinate selection and PCA are not repeated within folds. Validation folds therefore influence the reduced representations, including through their labels for L1 selection. Cross-validation scores may consequently be optimistic. The test data remain excluded from these fitting steps.

## 2.6 Metrics and variability

We report test accuracy and macro-F1:

$$\text{Accuracy} = \frac{1}{n_{\text{test}}} \sum_{i=1}^{n_{\text{test}}} \mathbf{1}\{\hat{y}_i = y_i\},$$

$$\text{MacroF1} = \frac{F1_0 + F1_1}{2},$$

where $F1_c$ is the F1 score treating class $c$ as the positive class. For either metric $M$, loss relative to the full representation is

$$\Delta M_m(k) = M_{\text{full}} - M_m(k),$$

where $m$ denotes the reduction method. Positive values indicate lower performance. Differences in proportions are multiplied by 100 when reported in percentage points.

For random selection, we report the mean and sample standard deviation across the 20 subsets at each size. Figure bands show one standard deviation and describe variability across coordinate draws on fixed data, rather than uncertainty over new training or test samples. The full baseline, L1 selection, and PCA each have one fitted configuration per size; their variability across data samples is not estimated.

# 3 Results

## 3.1 Exploratory data analysis

Figure 1 shows that negative reviews are generally longer than positive reviews, with median lengths of 98 and 84 tokens, respectively. Only 0.17% of training reviews exceed the 256-token limit, so truncation affects a small fraction of the sample.

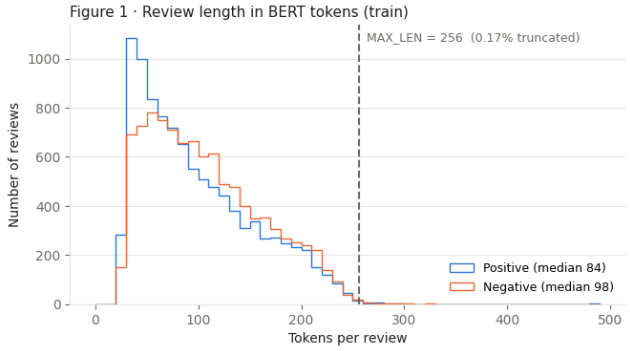



**Figure 1:** Training-review token lengths by sentiment. The dashed line marks the 256-token limit.

Figure 2 shows variation in coordinate scales, motivating standardization before selection and PCA. Most coordinate pairs have modest correlations, with a median absolute correlation of 0.084. Sentiment association varies across coordinates: 362 have absolute label correlations above 0.1, including 111 above 0.2. Thus, marginal sentiment association is spread across many coordinates, although its strength differs substantially.

Figure 3 shows that the first 100 principal components explain approximately 79% of standardized embedding variance, increasing to about 90% with 200 components. Among the first 50 components, PC4 has the strongest absolute label correlation, approximately 0.47. These results suggest that PCA can preserve substantial variation in fewer dimensions, while

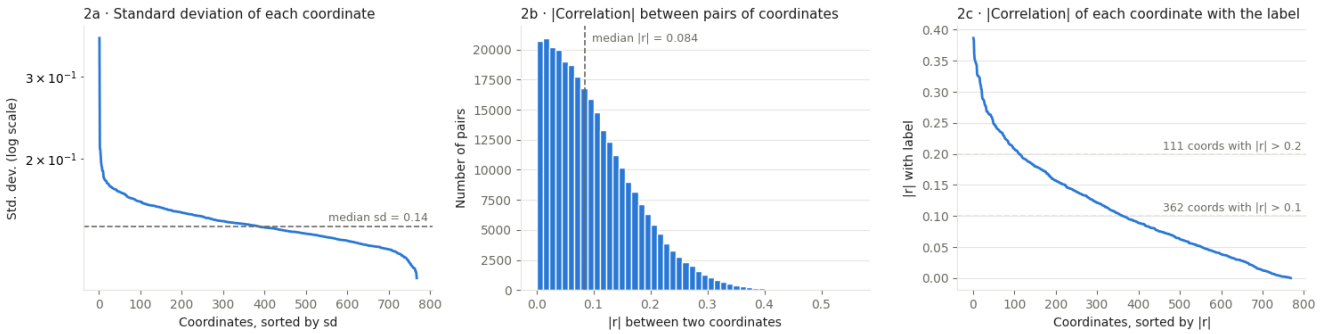



**Figure 2:** Training-embedding coordinate standard deviations, absolute pairwise correlations, and absolute correlations with sentiment labels.

also showing that the highest-variance component is not necessarily the most associated with sentiment.

### 3.2 Classification performance

The full 768-dimensional representation achieves 90.80% accuracy and approximately 0.9080 macro-F1. Figure 4 shows that performance improves with representation size for all three reduction methods. Accuracy and macro-F1 follow nearly identical patterns. Table 3 summarizes test accuracy.

Both L1 selection and PCA outperform average random selection at every tested size. The differences are largest under aggressive reduction. At $k = 25$, L1 selection achieves 85.92% accuracy compared with 73.36% for random selection, an advantage of 12.56 percentage points. PCA performs slightly better than L1 at $k = 25$ and $k = 50$, while L1 has higher observed accuracy at $k = 100$, 200, and 400.

### 3.3 Performance loss relative to the full representation

L1 selection preserves most of the baseline performance using a smaller coordinate subset. At $k = 100$, it retains approximately 13% of the coordinates and achieves 89.96% accuracy, a loss of 0.84 percentage points. At $k = 200$, it retains approximately 26% and achieves 90.68%, 0.12 percentage points below the baseline.

The gains diminish as more coordinates are retained. Increasing the L1 subset from 100 to 200 coordinates improves accuracy by 0.72 percentage points, whereas increasing it from 200 to 400 adds only 0.02 points. PCA also approaches the baseline, reaching 90.51% accuracy with 400 components. These are close observed scores.

### 3.4 Variability across random subsets

Random-selection accuracy increases from 73.36% at $k = 25$ to 90.16% at $k = 400$, while its standard deviation decreases from 2.22 to 0.15 percentage points. Thus, smaller subsets are more sensitive to which coordinates are selected. Larger random subsets provide more consistent performance and approach the full baseline, although they remain below L1 selection and PCA on average.

## 4 Discussion

### 4.1 Interpretation of the findings

The results show that a smaller representation can preserve much of the sentiment-classification performance of the full BERT embedding. However, the choice of retained coordinates matters. L1 selection outperforms average random selection at every tested size, with the largest advantages at smaller sizes. This suggests that supervised selection can identify

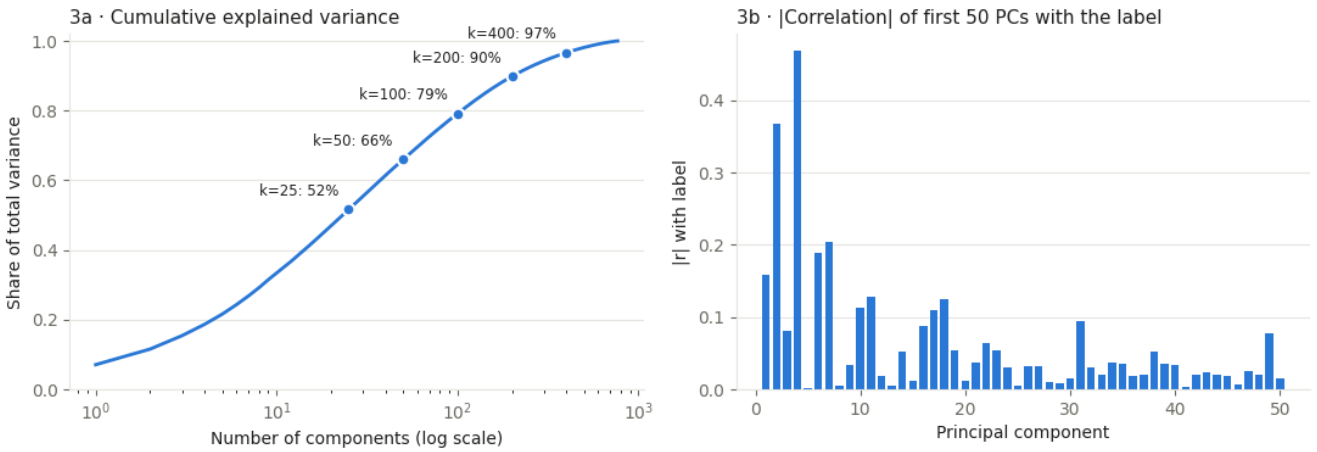



**Figure 3:** Cumulative explained variance from PCA on standardized training embeddings and absolute label correlations of the first 50 principal components.

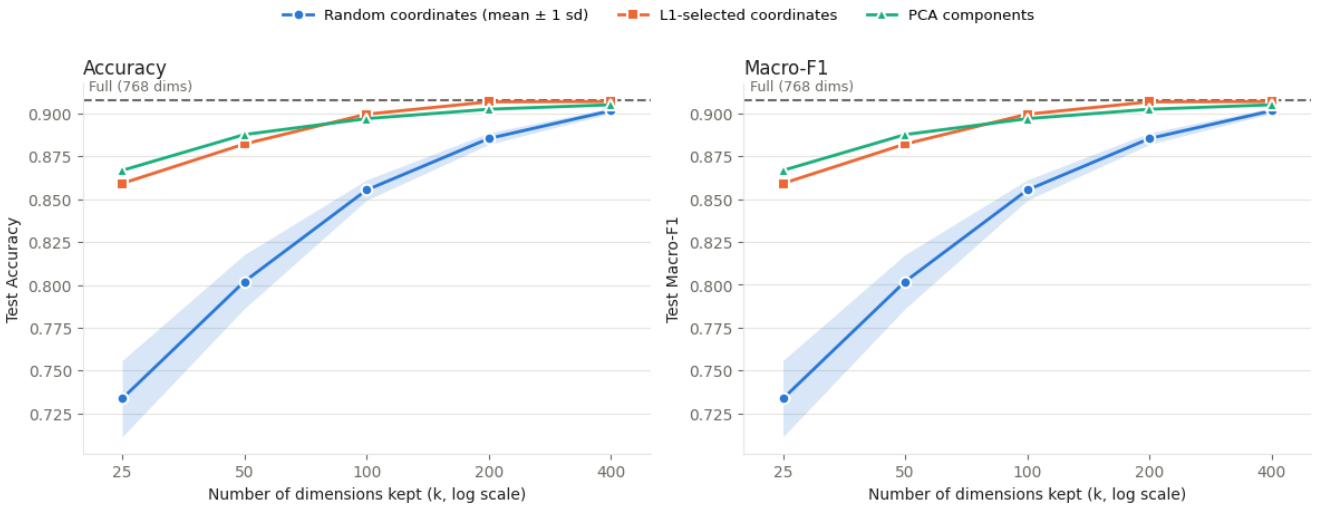



**Figure 4:** Test accuracy and macro-F1 versus representation size. The dashed line marks the full 768-dimensional baseline. Shaded bands show one standard deviation across 20 random coordinate subsets.

coordinates that provide more useful predictive information than an arbitrary subset.

The EDA and classification results also suggest that distributed sentiment information and predictive redundancy can coexist. Many coordinates are individually associated with the label, yet a selected subset of 200 coordinates approaches the full baseline. This does not mean that the embedding vectors themselves are sparse or that the remaining coordinates contain no useful information. Their contributions may overlap with those of retained coordinates or be less useful to the linear classifier on this task. Coordinate selection also depends on the embedding basis, so the selected coordinates should not be interpreted as a universal set of sentiment features.

PCA provides a competitive alternative. It has higher observed accuracy than L1 selection at 25 and 50 dimensions, while L1 is slightly ahead at larger sizes. PCA can combine information from many coordinates into a small number of components, which may help under aggressive reduction. However, PCA preserves variance rather than directly optimizing prediction. The EDA illustrates this distinction: PC4 has a stronger label correlation than PC1. The small performance differences between PCA and

**Table 3:** Test accuracy (%) by representation size. Random selection is reported as mean ± sample standard deviation in percentage points across 20 subsets. Bold values indicate the highest accuracy at each reduced size.

| Dimensions ($k$) | Random selection | L1 selection | PCA |
|---|---|---|---|
| 25 | $73.36 \pm 2.22$ | 85.92 | **86.68** |
| 50 | $80.19 \pm 1.57$ | 88.22 | **88.77** |
| 100 | $85.54 \pm 0.59$ | **89.96** | 89.70 |
| 200 | $88.54 \pm 0.34$ | **90.68** | 90.25 |
| 400 | $90.16 \pm 0.15$ | **90.70** | 90.51 |
| Full (768) | 90.80 | | |

L1 therefore do not establish that either method is generally preferable.

### 4.2 Limitations and future work

The analysis uses one balanced sample from Amazon Polarity, one frozen BERT model, mean pooling, and a linear classifier. The findings concern this particular setup and may differ for other tasks, models, or pooling strategies. Exact-text deduplication also does not rule out near-duplicate reviews.

A methodological limitation is that L1 selection and PCA are fitted on the complete training set before classifier cross-validation. Validation folds therefore influence the representations used during tuning, making the validation scores potentially optimistic. The held-out test data remain excluded from fitting, but a cleaner procedure would refit standardization, selection, and PCA within each training fold.

The reported random-selection variability measures sensitivity to coordinate choice on fixed data. It does not measure uncertainty across new data samples, and the L1 and PCA experiments do not assess selection stability across samples. Consequently, close test accuracies should not be interpreted as statistical equivalence or evidence that small differences are reproducible.

Finally, representation reduction occurs after the full BERT embedding is computed. It can reduce downstream feature storage and classifier size, but this implementation does not reduce transformer inference cost or establish end-to-end speed gains. Future work should first improve the cross-validation procedure and repeat the analysis across data samples. Additional tasks and paired comparisons of test predictions would help assess how broadly the findings hold and how uncertain the observed performance gaps are.

## 5 Conclusion

Frozen BERT embeddings allow substantial representation reduction for Amazon Polarity sentiment classification. L1 selection retains 200 of 768 coordinates and achieves 90.68% accuracy, compared with 90.80% for the full representation. Both L1 selection and PCA outperform average random coordinate selection at every tested size, while PCA performs slightly better at the smallest sizes. These results support task-specific predictive redundancy and show that informed reduction preserves performance more effectively than random coordinate removal. They do not establish universal sparsity or an optimal dimension for transformer representations.

## References

[1] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova. BERT: Pre-training of Deep

Bidirectional Transformers for Language Understanding. In *Proceedings of NAACL-HLT*, pp. 4171–4186, 2019. doi:10.18653/v1/N19-1423.

[2] N. Durrani, H. Sajjad, F. Dalvi, and Y. Belinkov. Analyzing Individual Neurons in Pre-trained Language Models. In *Proceedings of EMNLP*, pp. 4865–4880, 2020. doi:10.18653/v1/2020.emnlp-main.395.

[3] F. Dalvi, H. Sajjad, N. Durrani, and Y. Belinkov. Analyzing Redundancy in Pretrained Transformer Models. In *Proceedings of EMNLP*, pp. 4908–4926, 2020. doi:10.18653/v1/2020.emnlp-main.398.

[4] R. Tibshirani. Regression Shrinkage and Selection via the Lasso. *Journal of the Royal Statistical Society: Series B (Methodological)*, 58(1):267–288, 1996. doi:10.1111/j.2517-6161.1996.tb02080.x.

[5] I. T. Jolliffe and J. Cadima. Principal component analysis: a review and recent developments. *Philosophical Transactions of the Royal Society A*, 374:20150202, 2016. doi:10.1098/rsta.2015.0202.

[6] X. Zhang, J. Zhao, and Y. LeCun. Character-level Convolutional Networks for Text Classification. In *Advances in Neural Information Processing Systems*, vol. 28, pp. 649–657, 2015. Publisher's paper page.

# A Implementation and Reproducibility

## A.1 Software and execution settings

The experiments were implemented in Python using PyTorch, Transformers, Datasets, NumPy, pandas, and scikit-learn. The saved run records scikit-learn version 1.6.1 and CUDA-based embedding extraction. BERT inference uses float16 autocasting, while the resulting embeddings are stored as float32 arrays.

The reported experiment uses the complete sampled dataset, with `EMB_LIMIT=None` and `QUICK=False`. Data sampling uses seed 0, while the random coordinate generator, cross-validation splits, and PCA use seed 42. The random coordinate generator is initialized once and produces subsets sequentially across sizes and repetitions.

Data samples and embeddings are cached and reused when their files already exist. Changes to the dataset, model, or embedding settings therefore require regenerating the corresponding cached files.

## A.2 Code and supporting materials

The implementation is provided in `stat236_embedding.ipynb`. Code and supporting materials are available at:

`https://github.com/leocwq/STAT236`

The notebook saves individual experiment results to `results/full/results_raw.csv` and aggregated performance to `results/full/summary.csv`. Embedding settings and token length statistics are stored in `embeddings/full/meta.json`. Exploratory summaries and figures are written to `results/full/eda/`.

# B Supplementary Results

## B.1 Selected classifier hyperparameters

Table 4 reports the downstream L2 classifier's selected inverse regularization parameter $C$. These values are distinct from the L1 selector settings reported in the methodology section. Each representation is tuned separately using the same cross-validation procedure. The full 768-dimensional baseline selects $C = 0.01$.

**Table 4:** Selected downstream L2 classifier parameters. Values are rounded from the saved notebook outputs.

| $k$ | L1-selected features | PCA features |
|---|---|---|
| 25 | 0.0316 | 0.0316 |
| 50 | 0.00316 | 10 |
| 100 | 0.00316 | 0.00316 |
| 200 | 0.316 | 0.01 |
| 400 | 0.01 | 0.00316 |

## B.2 Range of random-subset performance

Table 5 supplements the mean and standard deviation reported in the main results with the minimum and maximum accuracy across the 20 random subsets at each size. The observed range narrows as more coordinates are retained, consistent with the declining standard deviation.

**Table 5:** Minimum and maximum test accuracy (%) across 20 random coordinate subsets at each size.

| $k$ | Minimum | Maximum |
|---|---|---|
| 25 | 68.33 | 76.28 |
| 50 | 76.98 | 82.74 |
| 100 | 83.95 | 86.43 |
| 200 | 87.98 | 89.07 |
| 400 | 89.93 | 90.45 |

These ranges describe variation across coordinate subsets on the same training and test samples. They do not provide confidence intervals for performance on new samples.